



Génie Logiciel

CER | Design Pattern

Mercredi 28 janvier 2026

TABLE DES MATIÈRES

Étude de cas : Design Pattern	2
AAV	2
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	3
Livrables	4
Pistes de solutions	4
Plan d'actions	4
Démarche : Design Pattern	5
Définition: Mots clés	5
Analyse et résolution : Design Pattern	7
Analyse	7
Solutions proposées	7
Conclusion	7
Capitalisation	8
Ressources	9

ÉTUDE DE CAS : DESIGN PATTERN

AAV

- 1 Identifier les catégories de Design Pattern (Création, Structurel, Comportement)
- 1 Décrire obligatoirement les cinq Design Pattern suivants Singleton, Factory, Observer, Bridge, Strategy
- 1 Pratiquer les Design Pattern suivants Singleton, Factory, Observer, Bridge, Strategy
- 1 Identifier les différents types de diagrammes UML permettant de réaliser la conception et le design d'une application
- 1 Connaître les formalismes UML permettant de décrire les concepts de la programmation orientée objet : encapsulation, héritage, ...
- 3 Pratiquer les diagrammes de classes et de séquence

MOTS CLÉS

- Architecture logicielle
- UML Design Pattern
- Qualité logicielle
- Singleton
- Génie logiciel
- Usine
- Observateur
- Pont

CONTEXTE

IF Roaming souhaite optimiser ses pratiques de développement. Mathilde se renseigne donc sur les Design Patterns pour proposer des solutions.

PROBLÉMATIQUE

Comment optimiser les pratiques de développement ?

CONTRAINTES

- Coût de développement
- POO
- Bases du génie logiciel
- Réutilisation du code
- Coût de formation de l'équipe

GÉNÉRALISATION

- POO
- Génie logiciel
- Design Patterns

LIVRABLES

- Plan d'amélioration
 - Plan d'amélioration des pratiques de développement

PISTES DE SOLUTIONS

- Etudier les Design Patterns
- Comprendre les principes du génie logiciel
- Comprendre les concepts des architectures logicielles modernes
- Etudier l'UML

PLAN D' ACTIONS

- Etudier les Design Patterns
- Comprendre les principes du génie logiciel
- Comprendre les concepts des architectures logicielles modernes
- Etudier l'UML
- Proposer un plan d'amélioration des pratiques de développement
- Rédiger un rapport de recherche
- Préparer une présentation orale pour formation

DÉMARCHE : DESIGN PATTERN

DÉFINITION : MOTS CLÉS

Design Pattern : En informatique, et plus particulièrement en développement logiciel, un patron de conception (souvent appelé design pattern) est un arrangement caractéristique de modules, reconnu comme bonne pratique en réponse à un problème de conception d'un logiciel. Il décrit une solution standard, utilisable dans la conception de différents logiciels

- Les patrons de conception ne sont ni des patrons d'architecture ni des idiotismes de programmation.
- Un patron de conception est la meilleure solution connue à un problème de conception récurrent.
- Un patron d'architecture est la meilleure solution connue à un problème d'architecture récurrent. Il apporte des solutions sur la manière de concevoir l'organisation à grande échelle (architecture) d'un logiciel en faisant abstraction des détails. Il concerne la structure générale d'un logiciel, sa subdivision en unités plus petites, comporte des guides de bonnes pratiques et des règles générales qui ne peuvent pas être traduites directement en code source.
- Le patron de conception suggère un arrangement, une manière d'organiser des modules ou des classes. Il décrit une organisation de classes fréquemment utilisée pour résoudre un problème récurrent. Le patron de conception parle d'instances, de rôles et de collaboration.

Singleton : Le Singleton est un patron de conception (design pattern), appartenant à la catégorie des patrons de création, dont l'objectif est de restreindre l'instanciation d'une classe à un seul objet.

Usine : Le pattern Usine (Factory) fournit une interface pour créer des objets dans une superclasse, mais permet aux sous-classes de modifier le type d'objets qui seront créés.

Observateur : Le pattern Observateur définit une dépendance un-à-plusieurs entre des objets, de sorte que lorsque l'objet observé change d'état, tous ses observateurs sont automatiquement notifiés et mis à jour.

Pont : Le pattern Pont sépare l'abstraction d'une classe de son implémentation, permettant ainsi de modifier indépendamment les deux.

Stratégie : Le pattern Stratégie définit une famille d'algorithmes, encapsule chacun d'eux et les rend interchangeables. Il permet à l'algorithme de varier indépendamment des clients qui l'utilisent.

Architecture logicielle : L'architecture logicielle fait référence à la structure globale d'un système logiciel, y compris les composants, leurs interactions et les principes de conception qui guident leur organisation.

Qualité logicielle : La qualité logicielle désigne l'ensemble des caractéristiques d'un logiciel qui déterminent sa capacité à satisfaire les besoins et attentes des utilisateurs, ainsi que sa fiabilité, maintenabilité et performance.

Génie logiciel : Le génie logiciel est une discipline qui applique des principes d'ingénierie pour la conception, le développement, la maintenance et la gestion de logiciels de haute qualité.

UML (Unified Modeling Language) : UML est un langage de modélisation graphique à base de pictogrammes conçu comme une méthode normalisée de visualisation dans les domaines du développement logiciel et en conception orientée objet.

UML est destiné à faciliter la conception des documents nécessaires au développement d'un logiciel orienté objet, comme standard de modélisation de l'architecture logicielle. Les différents éléments représentables sont :

- activité d'un objet/logiciel ;
- acteurs ;
- processus ;
- schéma de base de données ;
- composants logiciels ;
- réutilisation de composants.

Il est également possible de générer automatiquement tout ou partie du code, par exemple en langage Java, à partir des documents réalisés.

Diagramme de classes : Un diagramme de classes est un type de diagramme UML qui représente les classes d'un système, leurs attributs, méthodes et les relations entre elles.

Diagramme de séquence : Un diagramme de séquence est un type de diagramme UML qui illustre comment les objets interagissent entre eux au fil du temps, en mettant en évidence l'ordre des messages échangés.

ANALYSE ET RÉOLUTION : DESIGN PATTERN

ANALYSE

L'analyse des besoins de 2F Roaming révèle plusieurs défis dans les pratiques de développement actuelles. L'absence d'une structure claire dans le code, le manque d'automatisation des tests et l'inefficacité dans la gestion des versions sont des obstacles majeurs à la productivité et à la qualité du logiciel. De plus, l'équipe de développement semble manquer de connaissances approfondies en génie logiciel et en design patterns, ce qui limite leur capacité à concevoir des solutions robustes et évolutives.

SOLUTIONS PROPOSÉES

Pour répondre aux défis identifiés, l'adoption de certains design patterns est recommandée :

- **Singleton** : Garantir qu'une classe critique (par exemple, gestionnaire de configuration ou de connexion) ne possède qu'une seule instance partagée dans toute l'application.
- **Factory** : Centraliser la création d'objets complexes ou dépendants du contexte, facilitant l'extension et la maintenance du code.
- **Observer** : Mettre en place un système de notification automatique entre modules (exemple : interface graphique réagissant à des changements de données).
- **Bridge** : Séparer l'abstraction de l'implémentation pour permettre l'évolution indépendante des fonctionnalités et des plateformes techniques.
- **Strategy** : Permettre de changer dynamiquement d'algorithme ou de comportement (exemple : différentes méthodes de tri ou de validation).

L'intégration de ces patterns favorisera la réutilisabilité, la flexibilité et la robustesse des solutions logicielles développées par l'équipe.

CONCLUSION

L'adoption des design patterns identifiés permettra à 2F Roaming d'améliorer significativement ses pratiques de développement. En structurant le code de manière plus cohérente et en facilitant la maintenance, l'équipe pourra non seulement accroître sa productivité, mais aussi garantir une meilleure qualité des logiciels livrés. La formation continue sur ces concepts sera essentielle pour assurer une mise en œuvre efficace et durable.

CAPITALISATION

AAV	Commentaire	Statut
Identifier les catégories de Design Pattern (Création, Structurel, Comportement)	Les trois grandes catégories de design patterns sont identifiées et expliquées avec des exemples pour chacune.	Acquis
Décrire obligatoirement les cinq Design Pattern suivants Singleton, Factory, Observer, Bridge, Strategy	Chaque pattern est défini, illustré par un exemple concret et ses avantages sont explicités.	Acquis
Pratiquer les Design Pattern suivants Singleton, Factory, Observer, Bridge, Strategy	Des exemples d'implémentation sont fournis pour chaque pattern, démontrant leur utilisation dans des cas réels.	Acquis
Identifier les différents types de diagrammes UML permettant de réaliser la conception et le design d'une application	Les principaux diagrammes UML (classes, séquence, cas d'utilisation, etc.) sont listés et leur utilité est expliquée.	Acquis
Connaître les formalismes UML permettant de décrire les concepts de la programmation orientée objet : encapsulation, héritage	Les concepts POO sont associés à leurs représentations UML, avec exemples de notation pour chaque concept.	Acquis
Pratiquer les diagrammes de classes et de séquence	Des diagrammes de classes et de séquence sont réalisés pour illustrer des scénarios concrets liés aux design patterns.	Acquis

RESSOURCES

- ...